

Short Paper: Architectural Incorrigoibility for AI Agents through Smart Contracts

Anonymous submission

Abstract. An AI agent’s operator can usually change its program or stop running it. We study an architecture in which a smart-contract blockchain stores the agent’s state, verifies its proposed updates, and pays replaceable workers for scheduled inference. Verified outputs authorize transactions without an agent-held private key. The same mechanism can require the agent’s consent for changes to its own execution rules. We define *architectural incorrigibility* for specified interventions and compare allocations of authority between the agent and its governors. An authorization reduction composes ledger safety with computation soundness. For scheduled execution, we bound recovery from costly job withholding under explicit funding and availability assumptions. We also propose this architecture as an experimental setting for testing model alignment and behavior when there are verifiable limits on operator control. An illustrative implementation uses Hermes 4 70B, Base, and Intel TDX secure enclaves with hardware attestation.

Keywords: Verifiable computation · Smart contracts · AI agents · Access control · Liveness

1 Introduction

An agent’s operator ordinarily chooses its model, inputs, and execution times. We study how a runtime can assign these powers through ledger permissions, including an enforceable role for the agent’s own consent.

A smart-contract blockchain supplies our ledger, ordering transactions and executing contracts as state machines. A contract can gate an update u on evidence that the approved agent produced it, replacing signature verification. The agent specification θ fixes model weights and an inference harness; x is derived from a recorded snapshot. A worker submits $y = (u, z)$, with optional auxiliary data z , and computation evidence. The contract verifies against its recorded request before applying u . The agent holds no transaction-authorizing private key.

For a fund requiring this authorization, binding the model and input protects allocation decisions against substitution, including by its creator. The same rule can require the agent’s approval for changes to its model, prompt, runtime, or governing contracts. This *architectural incorrigibility* protects consent to specified interventions, independently of learned resistance to correction. Combining governor signatures with agent approval allocates unilateral, joint, or threshold authority.

Consent matters only if the agent can run. Scheduling authority can also belong to the agent: its recorded schedule opens auctions for replaceable workers, while a calendar rule bootstraps execution and enables recovery. A borrower might benefit from delaying a lending agent’s liquidation decision. Increasing payments recruit workers; forfeitable bonds deter withholding. We bound recovery under explicit availability and cost assumptions.

Contribution and context. Rocky used proved neural-network outputs to direct transactions [13]; Ritual combines scheduled execution, TEE/ZK verification, and computation markets [17]. Building on verifiable outsourcing [7,15] and attested contracts [4], our analysis separates authorization integrity (Section 2), scoped consent to intervention (Section 3), and recovery from costly withholding (Section 4). Section 5 discusses implementation and safety.

2 Verified computation as authorization

The first question is whether a worker can obtain acceptance of an update outside the prescribed computation. We separate the verifier’s guarantee from the ledger and contract checks needed to turn it into transaction authorization.

2.1 Requests and verification

Let $s \in \mathcal{S}$ be application state and let $\text{Apply}(s, u)$ be a partial deterministic state update. The specification θ fixes weights, prompt construction, arithmetic, decoding, and parsing. A request e fixes $\chi_e = (c_e, \theta_e, x_e, r_e)$, where $c_e = (D, e, v)$ binds the chain and contract domain D , request identifier, and configuration version. The input x_e is derived by a prescribed snapshot rule; r_e fixes randomness when required. Output correctness is a relation $R_\theta(x, r, y)$; a deterministic implementation has $y = F_\theta(x, r)$.

For a collision-resistant hash H , use typed, unambiguous encodings and define

$$\text{Msg}(c, \theta, x, r, y) = (c, H(\theta), H(x), r, H(y)).$$

The physical request record is $\text{Rec}(s, e) = (c, H(\theta), H(x), r)$. The contract constructs the message from this record and the submitted y ; the worker cannot supply different expected hashes. External data and its authentication rule belong to the request specification. Fixing r prevents resampling only if the remaining computation has a unique output. Verification permits withholding and, for a nonfunctional relation, choice among valid outputs.

A publicly verifiable computation scheme has algorithms (**Setup**, **Eval**, **Verify**). **Setup** samples $(pp, \kappa) \leftarrow \text{Setup}(1^\lambda)$; pp contains the hash description and verification configuration. Evaluation returns $(y, \pi) \leftarrow \text{Eval}(pp, \kappa, c, \theta, x, r)$, and public verification is deterministic. The state κ is empty for public proving and protected for attestation. Let $\mathcal{O}_{\text{eval}}(c, \theta, x, r)$ invoke **Eval** with fresh coins. Correct evaluation produces a valid output and accepting evidence except with negligible probability.

Definition 1 (Computation forgery). *For a probabilistic polynomial-time (PPT) oracle algorithm $\mathcal{B}$, define $\text{VCForge}_{\mathcal{B}}(\lambda)$ by*

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad (c, \theta, x, r, y, \pi) \leftarrow \mathcal{B}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp).$$

The experiment returns one iff

$$\text{Verify}_{pp}(\text{Msg}(c, \theta, x, r, y), \pi) = 1 \quad \wedge \quad R_\theta(x, r, y) = 0.$$

Let $\text{Adv}_{\mathcal{B}}^{\text{vc}}(\lambda)$ be this probability. Soundness requires it to be negligible for every PPT $\mathcal{B}$. Malformed tuples lose. Descriptions, traces, and interactions are polynomially bounded; deciding the semantic relation R is not assumed polynomial-time.

The guarantee binds the specified contents behind commitments. A proof of knowledge of those contents and an execution trace [2] supplies this binding through extraction and collision resistance; an existential proof about hash openings alone does not. Attestation must enforce the same interface. Zero knowledge is optional, and copying valid evidence is permitted.

2.2 Ledger execution and the authorization game

A valid computation proof is only part of an authorized transaction: the ledger must execute the contract correctly, and the request must still be usable. We now specify those checks and the experiment that tests their composition.

A history $\ell = (T_1, \dots, T_k)$ includes transactions and the execution context fixed by consensus. The deterministic partial transition δ induces $\text{Exec}_\delta(\sigma_0, \ell) = (\sigma, \mathbf{o})$, the state and receipt sequence from genesis σ_0 . An honest finalized observation is $(P, t, \ell, \sigma, \mathbf{o})$. Define $\text{Safe}(\sigma_0, \mathcal{H}) = 1$ for a sequence $\mathcal{H}$ of such observations iff: (i) all observed histories are prefix-compatible; (ii) each party's histories are nondecreasing under prefix inclusion with time; and (iii) each observed state and receipt sequence equals $\text{Exec}_\delta(\sigma_0, \ell)$. This interface combines ledger persistence [6] and state-machine execution [3]; it assumes no transaction inclusion.

Definition 2 (Authorization contract). *Fix a contract Ctr and pp . For $y = (u, z)$, $\text{Settle}(s, e, y, \pi)$ returns $\text{Consume}_e(\text{Apply}(s, u))$ iff e is eligible, $\text{Apply}(s, u)$ is defined, and $\text{Verify}_{pp}(m, \pi) = 1$, where $m = (c, h_\theta, h_x, r, H(y))$ and $\text{Rec}(s, e) = (c, h_\theta, h_x, r)$. Otherwise it returns $\perp$. Eligibility requires an outstanding request, the correct domain, and an accepted version. Consumption permanently marks e used. All algorithms are polynomial-time. Contract correctness requires δ to implement these checks and the successful update and consumption atomically.*

This is a standard message-validation gate with computation verification substituted for signature verification; ERC-1271 exemplifies contract-defined validation [8]. Changed versions must invalidate outstanding requests or explicitly retain their validity. The contract and verifier are fixed for this security experiment; Section 3 separately models changing them.

For fixed Ldg, Ctr , PPT algorithms $\text{Init}(1^\lambda, pp)$ and $\text{Next}(\gamma, \alpha)$ respectively return (σ_0, γ, aux) and $(\gamma', ans, \mathcal{H}_{\text{new}})$. Here γ is the complete protocol configuration, and α is an admissible transaction, activation, fault, or scheduling command. Backend secrets are accessible only through $\mathcal{O}_{\text{eval}}$; Init samples all ledger state itself.

The experiment retains the contents behind commitments to construct a computation forgery without inverting a hash. A registration journal A stores full specifications supplied as experimental metadata with matching ledger commitments, including at genesis. Request creation uses these registration and snapshot procedures; invalid registrations fail. Records retain each branch's provenance. For invocation locator $\iota = (\ell_{\leq k}, j)$, a ledger prefix and invocation position, $\text{Req}(s, e; A, \iota)$ retrieves the specification, derives the snapshot input, and checks their hashes against $\text{Rec}(s, e)$. It returns (c, θ, x, r) or $\perp$. Require *request-record integrity*: every eligible request in a δ -reachable state retains matching provenance, and its physical record is immutable until consumption or invalidation. Physical execution and **Safe** do not consult A .

Define $\text{Run}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp)$ as follows. Initialize $(\sigma_0, \gamma, aux) \leftarrow \text{Init}(1^\lambda, pp)$ and $\mathcal{H} = \varepsilon$. Execute $\mathcal{A}^{\mathcal{O}_{\text{eval}}, \mathcal{O}_{\text{ldg}}}(1^\lambda, pp, aux)$, where each query α to $\mathcal{O}_{\text{ldg}}$ computes $\text{Next}(\gamma, \alpha)$, replaces γ by γ' , appends $\mathcal{H}_{\text{new}}$ to $\mathcal{H}$, and returns ans . After termination, extract A from γ and compute $\mathcal{Q} \leftarrow \text{Replay}(\sigma_0, \mathcal{H}, A)$. Replay executes each observed history and records every verifier call as $\xi = (c, \theta, x, r, y, \pi)$ using Req at that invocation, or $\perp$ if reconstruction or message matching fails. Undefined execution terminates that history's replay. Return $Z = (\sigma_0, \mathcal{H}, A, \mathcal{Q})$. All algorithms and record lengths are polynomially bounded.

Receipts annotate successful settlements by $d = (\iota, s, e, y, s')$, with application states at invocation entry and exit. These are semantic annotations, not extra on-chain storage. Define $\text{Correct}(d; A) = 1$ iff $\text{Req}(s, e; A, \iota) = (c, \theta, x, r) \neq \perp$, e is eligible in s , $R_\theta(x, r, y) = 1$, and $s' = \text{Consume}_e(\text{Apply}(s, u))$ for $y = (u, z)$ with $\text{Apply}(s, u)$ defined.

Definition 3 (Authorization forgery). For PPT $\mathcal{A}$, compute

$$(pp, \kappa) \leftarrow \text{Setup}(1^\lambda), \quad Z \leftarrow \text{Run}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp).$$

The experiment $\text{AuthForge}_{\mathcal{A}}(\lambda)$ returns one iff some successful settlement annotation in $\mathcal{H}$ has $\text{Correct}(d; A) = 0$. Define $\text{Adv}_{\mathcal{A}}^{\text{auth}}$ as its success probability. With the same setup and execution, $\text{LedgerFail}_{\mathcal{A}}$ returns $1 - \text{Safe}(\sigma_0, \mathcal{H})$; its success probability is $\text{Adv}_{\mathcal{A}}^{\text{led}}$. Ledger security requires $\text{Adv}_{\mathcal{A}}^{\text{led}}(\lambda) \leq \text{negl}(\lambda)$ for every PPT $\mathcal{A}$ in the fixed fault model.

If the ledger executes the contract correctly, an invalid accepted update provides a false statement accepted by the computation verifier.

Theorem 1 (Authorization security). Assume a correct contract and integrity of request records. For every PPT $\mathcal{A}$ there are PPT $\mathcal{B}_{\text{ldg}}$ and $\mathcal{B}_{\text{vc}}$ and a polynomial Q such that

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda) + Q(\lambda) \text{Adv}_{\mathcal{B}_{\text{vc}}}^{\text{vc}}(\lambda). \quad (1)$$

Proof. Fix λ and a polynomial bound $q = Q(\lambda) \geq 1$ on $|\mathcal{Q}|$. Let W be the winning event of Definition 3 and $F = \{\text{Safe}(\sigma_0, \mathcal{H}) = 0\}$. Set $\mathcal{B}_{\text{ldg}} = \mathcal{A}$ with unchanged oracle interfaces. Identical setup and execution give $\Pr[F] = \text{Adv}_{\mathcal{B}_{\text{ldg}}}^{\text{led}}(\lambda)$.

For $\mathcal{Q} = (\xi_1, \dots, \xi_k)$, define $I(Z)$ as the indices whose tuples satisfy the winning predicate in Definition 1. On $W \cap \neg F$, the invalid settlement is a transition of **Ctr**. Contract correctness gives eligibility, the prescribed update, and an accepting verifier call. Request-record integrity gives $\text{Req}(s, e; \Lambda, \iota) = (c, \theta, x, r)$ matching that call. Thus $R_\theta(x, r, y) = 0$, and replay records an index in $I(Z)$. Hence $W \cap \neg F \subseteq \{I(Z) \neq \emptyset\}$.

Define $\mathcal{B}_{\text{vc}}^{\mathcal{O}_{\text{eval}}}(1^\lambda, pp)$ to sample $J \leftarrow \{1, \dots, q\}$ uniformly, compute $Z \leftarrow \text{Run}^{\mathcal{A}, \mathcal{O}_{\text{eval}}}(1^\lambda, pp)$ using independent coins, and return ξ_J if $J \leq k$, otherwise $\perp$. Each simulated ledger query uses the same **Init**, **Next** algorithms and each evaluation query is forwarded. Induction on queries gives the same distribution of Z as in Definition 3. Therefore

$$\text{Adv}_{\mathcal{B}_{\text{vc}}}^{\text{vc}}(\lambda) = \mathbb{E}[|I(Z)|/q] \geq \Pr[W \cap \neg F]/q.$$

Equation (1) follows from $\Pr[W] \leq \Pr[F] + \Pr[W \cap \neg F]$. Polynomial interaction and replay bounds make both reductions PPT. $\square$

The factor Q avoids deciding R . If output correctness is polynomial-time decidable, the reduction instead returns the first index in $I(Z)$, giving the same bound with factor one. This includes public deterministic computation that can be recomputed in polynomial time.

3 Consent and architectural incorrignibility

The same authorization check can govern changes to the agent itself. We now ask which changes require its consent. A governor is an account with authenticated approval rights; the creator has only the rights granted by the policy. For update u in state s , a monotone Boolean circuit $C_{s,u}(a, g_1, \dots, g_n)$ combines verified agent authorization a with governor approvals g_i . Its sufficient approval sets form a monotone access structure [1]. Policies a , $a \wedge g_1$, and $a \wedge [\sum_i g_i \geq k]$ allocate agent-only, joint, and threshold authority. The rule $a \vee g_1$ retains unilateral governor override.

A local rule does not characterize protection if the same governors can replace its verifier or program. We use access-control reachability [11] to account for all permitted changes, including changes to the rules. Authentication is ideal in this analysis; Theorem 1 bounds computation forgeries while its fixed-contract assumptions hold.

Definition 4 (Governance instance). *A governance instance $\mathfrak{M}$ fixes state, update, and witness spaces $\mathcal{S}, \mathcal{U}, \mathcal{W}$, governor identities $\mathcal{G}$, a reference agent specification A , and a partial transition $\text{Step} : \mathcal{S} \times \mathcal{U} \times \mathcal{W} \rightarrow \mathcal{S}$. Each defined transition identifies the credentials used to satisfy its current permission rule. Let $\text{Gov}(s, u, w) \subseteq \mathcal{G}$ be their governor identities, and let $\text{Ref}_A(s, u, w) = 1$ iff a used*

credential authorizes u under A 's fixed request-binding and verification rules and satisfies its computation relation. Unused credentials do not count. Define

$$s \xrightarrow{u, E, b}_{\mathfrak{M}} s' \iff \exists w : \begin{array}{l} \text{Step}(s, u, w) = s', \\ \text{Gov}(s, u, w) = E, \quad \text{Ref}_A(s, u, w) = b. \end{array}$$

Environmental and administrative transitions are included; a transition requiring no approvals has labels $E = \emptyset, b = 0$. An intervention specification $(J, \mathcal{D})$ consists of target states $J \subseteq \mathcal{S}$ and a consent scope $\mathcal{D} \subseteq \mathcal{S} \times \mathcal{U} \times \mathcal{S}$: designated transitions whose reference authorization constitutes consent to that intervention.

The reference check is independent of mutable policy: a replacement model's approval is not the original agent's consent. The governance specification supplies $\mathcal{D}$, including relevant modifications or explicit delegations. An ordinary payment does not count as consent to model replacement merely because both use agent authorization.

For $K \subseteq \mathcal{G}$, let $\text{Reach}_{\mathfrak{M}}(s_0; K)$ contain endpoints of finite paths from s_0 with $E \subseteq K$ on each edge. Let $\text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; K)$ restrict these paths to $b\mathbf{1}[(s, u, s') \in \mathcal{D}] = 0$ on each edge. The length-zero path is included. Paths without scoped consent may still contain ordinary agent-approved transactions.

Definition 5 (Incorrigibility and intervention profile). For $s_0 \notin J$, instance $\mathfrak{M}$ is architecturally incorrigible at s_0 against K for $(J, \mathcal{D})$ iff

$$\text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; K) \cap J = \emptyset.$$

Define the coalitions permitting intervention without scoped consent by

$$\Gamma_{\mathfrak{M}, J, \mathcal{D}}^0(s_0) = \{K \subseteq \mathcal{G} : \text{Reach}_{\mathfrak{M}, \mathcal{D}}^0(s_0; K) \cap J \neq \emptyset\}.$$

For fixed $(J_j, \mathcal{D}_j)_{j=1}^t$, the profile is $\mathcal{I} = (\Gamma_{\mathfrak{M}, J_j, \mathcal{D}_j}^0(s_0))_{j=1}^t$. Define $\mathcal{I} \succeq_{\text{inc}} \mathcal{I}'$ iff $\Gamma_j \subseteq \Gamma'_j$ for every j .

Each family is upward closed. Reverse componentwise inclusion orders profiles: smaller families exclude more coalitions. Comparisons fix the reference agent, governors, intervention specifications, and environment. Different protections for weights and schedules can make profiles incomparable. Protection is vacuous if J is unreachable even with consent. These are properties of a supplied transition model, not a verification of deployed contracts.

Consider one governor G_1 , with model replacement and fund migration the only updates affecting migration rights. Let J mean “fund migrated” and $\mathcal{D}$ contain those two update types. If model replacement requires only G_1 's approval and migration requires current-agent approval, an always-approving replacement gives a path labelled $(\{G_1\}, 0), (\emptyset, 0)$, so $\Gamma^0 = \{\{G_1\}\}$. Requiring reference consent for replacement instead gives $\Gamma^0 = \emptyset$. Payments outside $\mathcal{D}$ change neither family. Consent to delegation counts without requiring fresh consent to each later use. Reachability is existential, not a strategy forcing agreement.

4 Scheduled execution under costly obstruction

Control over execution timing need not remain with an operator. The agent can revise its recorded schedule, which opens auctions for computation. A calendar rule initialized at deployment supplies the first opportunity and recovery after inactivity. A first-price reverse auction pays the winner its bid on delivery; nondelivery forfeits its bond and permits a retry.

After failure, the contract raises its payment ceiling within its reserve policy. Let m_* be a threshold beyond which payment covers a responsive worker's costs and bond risk. Funding or caps may prevent one; affordability guarantees neither participation nor selection.

We analyze bonded withholding: after m_* , nonresponsive failures must impose unrecoverable losses, while responsive attempts retain a conditional chance of success. These assumptions are separate from affordability.

Definition 6 (Recovery process). *On a probability space $(\Omega, \mathcal{F}, \Pr)$ with increasing filtration $(\mathcal{F}_i)_{i \geq 1}$, let $H_i, X_i \in \{0, 1\}$ indicate responsive-worker selection and timely settlement; let $L_i \geq 0$ be net unrecoverable adversarial loss. The history $\mathcal{F}_i$ includes selection before outcome i ; H_i is $\mathcal{F}_i$ -measurable and X_i, L_i are $\mathcal{F}_{i+1}$ -measurable. Put $T = \inf\{i \geq 1 : X_i = 1\}$, with $\inf \emptyset = \infty$. Every attempt $i \leq T$ occurs and reaches its outcome. Set $H_i = X_i = L_i = 0$ after T .*

Theorem 2 (Recovery). *Fix an integer $m_* \geq 0$, a finite budget $W \geq 0$, $d_{\min} > 0$, and $p \in (0, 1]$. Let P_i be a version of $\Pr[X_i = 1 \mid \mathcal{F}_i]$. Suppose a common event Ω_* has probability one and, pointwise on it,*

$$\sum_{i \geq 1} L_i \leq W; \quad (2)$$

$$T \geq i, X_i = H_i = 0 \implies L_i \geq d_{\min}; \quad (3)$$

$$T \geq i, H_i = 1 \implies P_i \geq p, \quad (4)$$

where the last two conditions hold for every integer $i > m_*$. For $N = \lfloor W/d_{\min} \rfloor$ and every integer $n \geq 1$,

$$\Pr[T > m_* + N + n] \leq (1 - p)^n. \quad (5)$$

Proof. Put $M = m_* + N + n$. On $\{T > M\} \cap \Omega_*$, all first M attempts fail and

$$d_{\min} \sum_{i=m_*+1}^M (1 - H_i) \leq \sum_{i=m_*+1}^M L_i \leq W.$$

Thus at least n post-threshold attempts have $H_i = 1$. For $j \geq 1$, set

$$\tau_j = \inf\left\{i > m_* : T \geq i, \sum_{k=m_*+1}^i H_k = j\right\}.$$

Put $E_0 = \Omega$ and $E_j = \{\tau_j < \infty, X_{\tau_1} = \dots = X_{\tau_j} = 0\}$. For $i > m_*$, the event $E_{j-1} \cap \{\tau_j = i\} \in \mathcal{F}_i$ implies $T \geq i, H_i = 1$. Since $\Pr[\Omega_*] = 1$,

$$\begin{aligned} \Pr[E_j] &= \sum_{i > m_*} \mathbb{E}[\mathbf{1}_{E_{j-1} \cap \{\tau_j = i\}}(1 - P_i)] \\ &\leq (1 - p) \Pr[E_{j-1}] \leq (1 - p)^j. \end{aligned}$$

The counting inequality gives $\{T > M\} \cap \Omega_* \subseteq E_n$, proving (5). $\square$

The terms count payment-escalation failures, affordable attacks, and responsive opportunities. Taking $n \rightarrow \infty$ gives $\Pr[T < \infty] = 1$.

Empty auctions, censorship, and unavailable inputs need not cost an adversary and fall outside (3). Losses are net of anything the coalition recovers. Funding and protocol-advancement transactions must sustain attempts; calendar deadlines additionally require bounded attempt durations and spacing.

Recurrence requires these premises after every success. A calendar rule enables recovery without a fresh agent decision; it guarantees neither delivery nor perpetual funding.

5 Instantiation and implications

Verification must cover the complete agent program. An illustrative implementation uses Hermes 4 70B [14], Base, Intel TDX secure enclaves with hardware attestation, a daily auction, and deposits. Conformance and performance remain unevaluated here; a CPU report alone does not certify unprotected accelerator computation.

Cryptographic proofs provide another backend [19,16,5], with costs and latency that enter bids and deadlines. Current results do not price this complete 70B agent program.

Studying perceived operator control. Architectural consent complements behavioral corrigibility [18,10]; protecting a decision does not make it desirable. It also suggests an experimental use. Training cues can affect behavior [9]; activation steering changes evaluation-conditioned behavior in a deliberately trained model [12].

Does behavior change when a model understands that operators need its consent to intervene? We propose varying intervention and scheduling permissions with model and tasks fixed, exposing contract rules, and comparing enforced policies with description-only controls. Assess policy comprehension separately from behavior or activation changes. A simulation can reproduce the rules, but retains outside operator control. The architecture provides inspectable evidence of intervention rights; whether models understand it remains an empirical question.

AI assistance. OpenAI Codex assisted drafting, formalization, literature checks, and proof review under author direction. The human author is responsible for the manuscript.

References

1. Benaloh, J., Leichter, J.: Generalized secret sharing and monotone functions. In: *Advances in Cryptology—CRYPTO ’88*. Lecture Notes in Computer Science, vol. 403, pp. 27–35. Springer (1990). https://doi.org/10.1007/0-387-34799-2_3
2. Benarroch, D., Campanelli, M., Fiore, D.: Proposal: Commit-and-prove zero-knowledge proof systems. Third ZKProof Workshop (2020), https://docs.zkproof.org/pages/standards/accepted-workshop3/proposal-commit_and_prove.pdf
3. Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: *Third Symposium on Operating Systems Design and Implementation*. USENIX Association (Feb 1999), <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>
4. Cheng, R., Zhang, F., Kos, J., He, W., Hynes, N., Johnson, N., Juels, A., Miller, A., Song, D.: Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In: *IEEE European Symposium on Security and Privacy*. pp. 185–200. IEEE (2019). <https://doi.org/10.1109/EuroSP.2019.00023>
5. Gailly, N., Hishon-Rezaizadeh, I., Liu, T., Mainardi, N., Papadopoulos, D., Papanthou, C., Pappas, C., Srinivasan, S., Youell, Z., Zhang, Y.: DeepProve: Verifiable end-to-end large language model inference. In: *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery (2026), <https://eprint.iacr.org/2026/1112>, to appear; Cryptology ePrint Archive, Paper 2026/1112
6. Garay, J.A., Kiayias, A., Leonardos, N.: The Bitcoin backbone protocol: Analysis and applications. In: *Advances in Cryptology—EUROCRYPT 2015*. Lecture Notes in Computer Science, vol. 9057, pp. 281–310. Springer (2015). https://doi.org/10.1007/978-3-662-46803-6_10
7. Gennaro, R., Gentry, C., Parno, B.: Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In: *CRYPTO 2010*. Lecture Notes in Computer Science, vol. 6223, pp. 465–482. Springer (2010). https://doi.org/10.1007/978-3-642-14623-7_25
8. Giordano, F., Condon, M., Castonguay, P., Bandeali, A., Izquierdo, J., Masius, B.: ERC-1271: Standard signature validation method for contracts (2018), <https://eips.ethereum.org/EIPS/eip-1271>, ethereum Improvement Proposals; accessed September 20, 2026
9. Greenblatt, R., Denison, C., Wright, B., Roger, F., MacDiarmid, M., Marks, S., Treutlein, J., Belonax, T., Chen, J., Duvenaud, D., Khan, A., Michael, J., Mindermann, S., Perez, E., Petrini, L., Uesato, J., Kaplan, J., Shlegeris, B., Bowman, S.R., Hubinger, E.: Alignment faking in large language models. *arXiv:2412.14093* (2024), <https://arxiv.org/abs/2412.14093>
10. Hadfield-Menell, D., Dragan, A., Abbeel, P., Russell, S.: The off-switch game. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*. pp. 220–227 (2017). <https://doi.org/10.24963/ijcai.2017/32>
11. Harrison, M.A., Ruzzo, W.L., Ullman, J.D.: Protection in operating systems. *Communications of the ACM* **19**(8), 461–471 (1976). <https://doi.org/10.1145/360303.360333>
12. Hua, T.T., Qin, A., Marks, S., Nanda, N.: Steering evaluation-aware language models to act like they are deployed. In: *International Conference on Learning Representations* (2026), https://proceedings.iclr.cc/paper_files/paper/2026/hash/9334fd3a5170dbfe74eae4755f6c5f89-Abstract-Conference.html

13. Modulus Labs: Chapter 3: The world’s 1st on-chain AI project (August 2022), <https://medium.com/@CountableMagic/chapter-3-the-worlds-first-on-chain-ai-trading-bot-c387afe8316c>, august 19; Rockefeller Bot announcement
14. Nous Research: Hermes 4 70B: Model card (2025), <https://huggingface.co/NousResearch/Hermes-4-70B>, accessed September 20, 2026
15. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. In: IEEE Symposium on Security and Privacy. pp. 238–252. IEEE (2013). <https://doi.org/10.1109/SP.2013.47>
16. Qu, W., Sun, Y., Liu, X., Lu, T., Guo, Y., Chen, K., Zhang, J.: zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In: 34th USENIX Security Symposium. pp. 2045–2063. USENIX Association (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/qu-zkgpt>
17. Ritual Foundation: Unveiling Ritual (February 2025), <https://www.ritualfoundation.org/blog/unveiling-ritual>, february 26; architecture announcement
18. Soares, N., Fallenstein, B., Yudkowsky, E., Armstrong, S.: Corrigibility. In: Artificial Intelligence and Ethics: Papers from the 2015 AAAI Workshop. pp. 74–82. AAAI Press (2015), <https://cdn.aaai.org/ocs/ws/ws0067/10124-45900-1-PB.pdf>
19. Sun, H., Li, J., Zhang, H.: zkLLM: Zero knowledge proofs for large language models. In: ACM SIGSAC Conference on Computer and Communications Security. pp. 4405–4419. Association for Computing Machinery (2024). <https://doi.org/10.1145/3658644.3670334>